

# 第5回: FastAPIでChatバックエンドを作る

LLMアプリケーション基礎

## 今日のゴール

---

OpenAI API を **FastAPI** でラップして、HTTP 経由で AI に話しかけられる状態を作る

# 今日の流れ

---

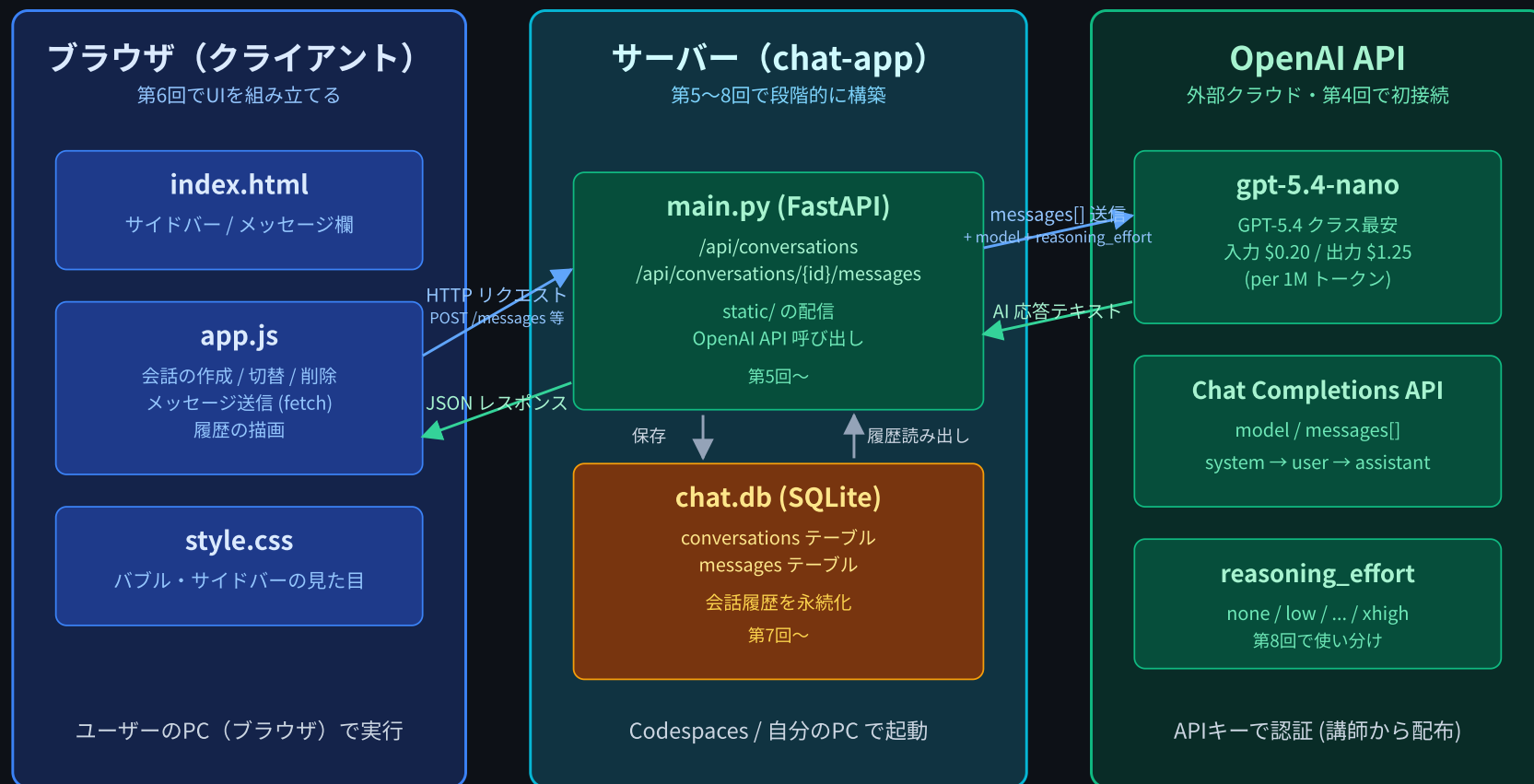
## 前半

- 前回までのおさらい
- なぜバックエンド経由にするか（APIキー保護）
- `POST /api/chat` の設計と Pydantic 型定義

## 後半

- 実装（OpenAIクライアント / エラーハンドリング / CORS）
- Swagger UI で動作確認
- 演習: `/api/chat` を完成させる

# chat-app の全体像



今日は中央の **main.py (FastAPI)** を立ち上げ、OpenAI 呼び出しを HTTP でラップする

## 前回までのおさらい

第4回: Python から OpenAI API を直接呼んだ

- ターミナルで `python chat.py` を実行
- `client.chat.completions.create(...)` で1往復
- `gpt-5.4-nano` + `reasoning_effort` で挙動を切り替え
- APIキーは **シェルの環境変数** で渡す (`export OPENAI_API_KEY=sk-...`)

→ 動いた。でもこれは「自分のPC専用」だった

# 今日やること

## 第5回のゴール

- **HTTP** のサーバ にして、外から呼べる形にする
- `POST /api/chat` というエンドポイントを設計
- Pydantic でリクエスト/レスポンスの **型** を定義
- 環境変数からキーを読む
- API失敗時のエラーハンドリング
- Swagger UI で動作確認
- **演習**: 自分の手でバックエンドを完成させる

# なぜわざわざバックエンド経由にするのか

理由は1つ。 **APIキー保護** です

ブラウザから直接 OpenAI を呼ぶ場合を考えてみる

```
// !!! 絶対にやってはいけない例 !!!  
fetch("https://api.openai.com/v1/chat/completions", {  
  headers: { "Authorization": "Bearer sk-proj-..." }, // ← キーが丸見え  
  ...  
});
```

このJSはブラウザに配信される = 誰でも開発者ツールで読める

## APIキーが漏れるとどうなるか

- 第三者が あなたのキーで API を叩き放題
- 課金は当然あなた持ち
- 月予算上限を超えて停止 → 自分のアプリも止まる
- 最悪、巨額の請求 (上限を設定していなかった場合)

OpenAI は漏洩したキーを 検知して自動で無効化 することもあるが、それより前に攻撃が走っている可能性が高い

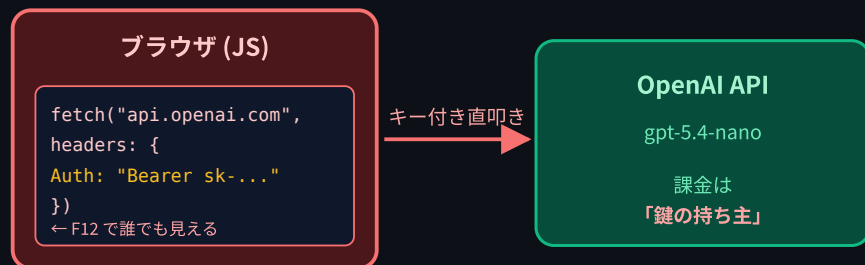
→ キーは絶対にフロント (ブラウザ) に出さない



# 解決策: バックエンドを挟む

## なぜバックエンドを挟むのか — APIキー保護

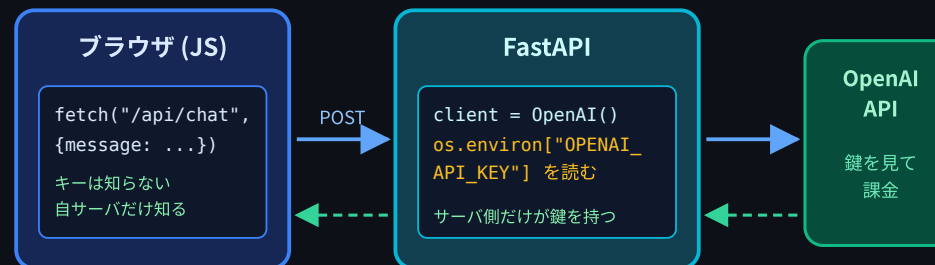
⊗ 危険: ブラウザから直接 OpenAI を呼ぶ



△ 結果

第三者がキーを抜いて API叩き放題 → 課金は あなた持ち  
月予算上限を超えるとアプリ停止 / 最悪は巨額請求

⊗ 正しい: 自分のバックエンドを挟む



⊗ 結果

ブラウザは OpenAI を知らない / キーはサーバの環境変数のみ  
レート制限・ログ・監視・認証もこのレイヤーで挟める

原則: 「鍵は ブラウザに出さない。サーバの 環境変数 にだけ存在する」

①コードに直書きしない ②`git push` 前に `git status` で確認 ③ Network タブで /api/chat に Authorization が無いことを確認

「APIキー保護①」= そもそもコミットしない / 「APIキー保護②」= バックエンドで持つ (今日のテーマ)

# POST /api/chat の設計

最小設計から始める

- メソッド: `POST` (副作用がある + ボディを送るため)
- パス: `/api/chat`
- リクエストボディ: `{ "message": "こんにちは" }`
- レスponseボディ: `{ "reply": "こんにちは!何かお手伝いできますか?" }`

ポイント

- 第5回時点では 会話履歴を持たない (毎回単発の質問応答)
- マルチターン (履歴を積む) は次回 (第6回) の話

# Pydantic で型を定義する

FastAPI は Pydantic と組み合わせて 自動でバリデーション してくれる

```
from pydantic import BaseModel

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str
```

これだけで

- `message` が無い → 自動で 422 エラー
- `message` が文字列じゃない → 自動で 422 エラー
- 型情報が Swagger UI に自動で載る

# エンドポイントの骨格

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/api/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    # 1. OpenAI API を呼ぶ
    # 2. 返答テキストを取り出す
    # 3. ChatResponse に詰めて返す
    ...
```

- `request: ChatRequest` と書くだけで、FastAPI が JSON をパースして型チェックする
- `response_model` を指定すると、レスポンスの型も保証される

# OpenAI クライアントの初期化

```
from openai import OpenAI

# 環境変数 OPENAI_API_KEY を自動で読み取る
client = OpenAI()

MODEL_NAME = "gpt-5.4-nano"
REASONING_EFFORT = "low" # チャット用途は none～low で十分
```

## ポイント

- `OpenAI()` は引数を渡さなければ `OPENAI_API_KEY` を勝手に読む
- だから起動前に必ず `export OPENAI_API_KEY=sk-...` しておく
- コード内に キーを直書きしない

## system プロンプトは固定で持つ

第5回では「常に親切なアシスタント」になってもらう

```
SYSTEM_PROMPT = (  
    "あなたは親切で丁寧なアシスタントです。日本語で回答してください。"  
)
```

- 第8回で「会話ごとに system プロンプトを変える」話に発展する
- 今日は **コード内の定数** として持つ
- system プロンプトの考え方は第2回でやった通り (役割の指示)

## 中身の実装

```
@app.post("/api/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        response = client.chat.completions.create(
            model=MODEL_NAME,
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": request.message},
            ],
            reasoning_effort=REASONING_EFFORT,
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"AI APIの呼び出しに失敗: {e}")

    reply = response.choices[0].message.content
    return ChatResponse(reply=reply)
```

# エラーハンドリングの考え方

OpenAI API は 失敗することがある

- ネットワーク断
- APIキーが無効・期限切れ
- レート制限 (429)
- OpenAI 側の障害

何もしないと FastAPI は 500 を返すが、 詳細は隠れる

明示的に `try/except` で `HTTPException(500, detail=...)` に変えると、クライアント (フロント) 側でエラー内容を扱いやすくなる

```
try:
    response = client.chat.completions.create(...)
except Exception as e:
    raise HTTPException(status_code=500, detail=f"AI APIの呼び出しに失敗: {e}")
```



# CORS を全許可しておく

開発中はフロントとバックエンドのオリジンが違ふことがある

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- 本番では絞るべきだが、今は学習用なので全許可
- これが無いとブラウザの fetch が CORS エラーで弾かれる

## 静的ファイル配信もしておく

第6回でフロントを置く場所として `static/` を用意

```
from fastapi.staticfiles import StaticFiles

app.mount("/", StaticFiles(directory="static", html=True), name="static")
```

- `/` を開くと `static/index.html` が返る
- 今日は プレースホルダ だけ置いておく
  - 「フロントは第6回で作ります。Swagger UI からテストしてください」と表示

# 起動方法

## 毎回の流れ

```
# 1. APIキーを環境変数にセット (ターミナルを開き直したら毎回必要)  
export OPENAI_API_KEY=sk-...  
  
# 2. 起動  
python main.py
```

## main.py の末尾

```
if __name__ == "__main__":  
    import uvicorn  
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

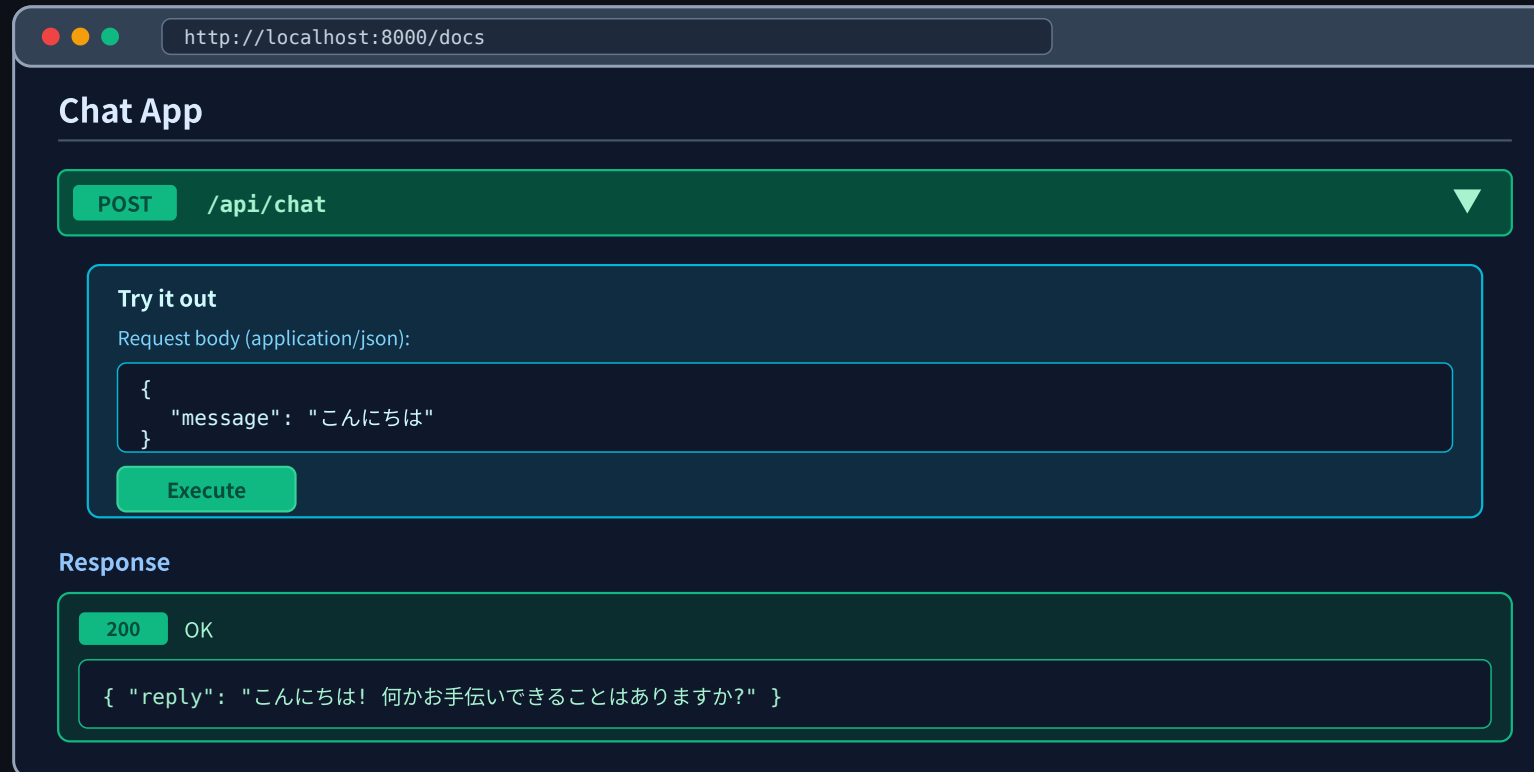
→ <http://localhost:8000> でサーバが立ち上がる

# Swagger UI で動作確認

FastAPI は 自動で API ドキュメント を作ってくれる

<http://localhost:8000/docs> を開くと...

Swagger UI: <http://localhost:8000/docs>



## Swagger UI のテスト手順

1. ブラウザで <http://localhost:8000/docs> を開く
  2. `POST /api/chat` をクリックして展開
  3. **Try it out** ボタンを押す
  4. Request body を編集: `{ "message": "Pythonって何?" }`
  5. **Execute** を押す
  6. 下の Response 欄に AIの返答が表示される
- ターミナルにも OpenAI API へのリクエストログが流れるはず

# 設計の確認: フロントにキーは渡らない?

実装が出来たら 設計を見直す

チェックポイント

- [] `main.py` の中に `sk-` から始まる文字列は無いかな?
- [] `static/` の中 (HTML/CSS/JS) に APIキーは無いかな?
- [] `git status` でうっかり `.env` 等をコミットしようとしていないかな?
- [] ブラウザの開発者ツールの Network タブを開いて、  
`/api/chat` のリクエストヘッダに APIキーが入っていないかな?

ブラウザは 自分のバックエンドだけ を見ていればOK

# 演習

`session05/exercise/` をコピーするか、自分の chat-app に追加して

1. `POST /api/chat` を実装する ( `ChatRequest` / `ChatResponse` )
2. `export OPENAI_API_KEY=sk-...` してから `python main.py` で起動
3. <http://localhost:8000/docs> を開く
4. Swagger UI から **3パターン**くらい 質問を投げてみる
  - 雑談 / 計算問題 / Python の質問など
5. わざとキーを間違えて起動して、500 エラーが返ることを確認

余力があれば

- `reasoning_effort` を `high` に変えて応答の質・速さの違いを観察

# よくあるハマりどころ

- `OPENAI_API_KEY` が無いと言われる
  - `export` を忘れている / ターミナルを開き直して消えた
  - `echo $OPENAI_API_KEY` で確認
- ポート 8000 が使えない
  - 既に別プロセスが使っている → kill するか別ポート
- Swagger UI で 422 が返る
  - リクエストボディの JSON が間違っている可能性
  - `{ "message": "..."}`  の形式を確認
- 500 が返る
  - APIキーが無効・期限切れ・残高不足
  - ターミナルのログにエラー詳細が出ているはず



## 本日のまとめ

---

### 学んだこと

1. ブラウザから OpenAI を 直接呼んではいけない (APIキーが漏れる)
2. 解決策: バックエンドを挟む → キーはサーバ側にしか存在しない
3. **FastAPI + Pydantic** で型安全な `POST /api/chat` を実装
4. 環境変数からキーを読む / `try/except` で 500 を返す
5. **Swagger UI** ( `/docs` ) は実装中の動作確認に超便利
6. まだ会話履歴は持たない (毎回単発) → 次回マルチターン化

## 次回予告

### 第6回: チャットUI + マルチターン会話

ブラウザで動くチャット画面を作る。今日作った `/api/chat` を `fetch` で呼び、`messages` 配列に履歴を積んで AI が「前の発言を覚えている」状態を実現する。トークン と コンテキストウィンドウ の直感もここで扱う。

## 提出物

---

実習で作成したファイルをフォームから提出してください:

1. `session05/exercise/` の `main.py` の GitHub のURL

◦ 例: `https://github.com/ユーザー名/リポジトリ名/blob/main/session05/exercise/main.py`

お疲れ様でした！